Redes neuronais e PyTorch – consulta rápida

1 · De raiz, com NumPy

Módulo N3_04
notação: X, W, b, Z, A, ŷ, L, η

01 NEURÓNIO E ATIVAÇÕES

N3_04 · §1–2

`z = X @ w + b` `a = sigmoide(z)`	Soma ponderada + ativação. Com sigmoide, é uma regressão logística.
`sigmoide(z) = 1 / (1 + exp(-z))`	Saída 0–1; derivada σ(z)(1 − σ(z)), no máximo 0,25.
`np.tanh(z)`	Saída –1 a 1; derivada `1 − tanh²(z)`.
`np.maximum(0, z)`	ReLU: derivada 1 se z > 0, senão 0. A mais usada nas camadas escondidas.
`(X @ W1) @ W2 == X @ (W1 @ W2)`	Sem ativação não linear, várias camadas valem uma (fronteira reta: não resolve o XOR).

02 PERDA E DESCIDA DO GRADIENTE

N3_04 · §3

`-mean(y*log(p) + (1-y)*log(1-p))`	Entropia cruzada binária: castiga erros confiantes. Usar np.clip(p, 1e-12, 1 – 1e-12).
`w = w - taxa * dL_dw`	Descida do gradiente: um passo contra a inclinação.
Taxa de aprendizagem	Pequena: aprende devagar. Grande: ultrapassa o mínimo e diverge.

03 UMA CAMADA ESCONDIDA: FORMAS

N3_04 · §4

Passo	Cálculo	Forma
entrada	`X`	`(n, 2)`
escondida	`Z1 = X @ W1 + b1`	`(n, h)`
ativação	`A1 = relu(Z1)`	`(n, h)`
saída	`Z2 = A1 @ W2 + b2`	`(n, 1)`
probabilidade	`A2 = sigmoide(Z2)`	`(n, 1)`

`rng.normal(0, np.sqrt(2 / n_ent), size=(n_ent, h))`	Inicialização de He (ReLU). Nunca tudo igual: os neurónios ficariam cópias.

04 RETROPROPAGAÇÃO

N3_04 · §5

`dZ2 = (A2 - y) / n`
`dW2 = A1.T @ dZ2;  db2 = dZ2.sum(0)`
`dA1 = dZ2 @ W2.T`
`dZ1 = dA1 * relu_deriv(Z1)`
`dW1 = X.T @ dZ1;  db1 = dZ1.sum(0)`
	Regra da cadeia, de trás para a frente. `*` é elemento a elemento; `@` é produto de matrizes. Cada gradiente tem a forma do seu parâmetro.

`+ l2 * W`	Regularização L2: somar ao gradiente de cada W.
`(L(w + ε) - L(w - ε)) / (2ε)`	Verificar os gradientes: erro relativo < 1e-6 (usar tanh, sem o «bico» da ReLU).

05 TREINAR

N3_04 · §6

`for epoca in range(epocas):`
`  for Xl, yl in lotes(X, y, 32):`
`  g = rede.gradientes(Xl, yl)`
`  for k, p in rede.params.items():`
`  p -= taxa * g[k]`
	Época = uma passagem pelos dados de treino. Os mini-lotes vêm de um gerador baralhado (cap. 25).

Generaliza bem	Perdas de treino e de validação descem juntas.
Sobreajuste	A de validação sobe, mesmo com exatidão estável: a rede fica confiante demais nos erros.
Remédios	L2, paragem antecipada (repor os melhores pesos) e, sobretudo, mais dados.

06 DO NUMPY AO PYTORCH

N3_04 · §7

À mão (N3_04)	PyTorch (N3_05)
`params, inicialização`	`nn.Linear(2, 16)`
`entropia_cruzada`	`nn.BCEWithLogitsLoss()`
`gradientes(...)`	`perda.backward()`
`p -= taxa * g`	`otimizador.step()`
`mini_lotes (gerador)`	`DataLoader(..., shuffle=True)`

Acompanha sempre a forma (`.shape`) de cada matriz: quase todos os erros numa rede são formas que não batem certo.

Redes neuronais e PyTorch – consulta rápida 2 · PyTorch

07 TENSORES

N3_05 · §1

`torch.tensor(X, dtype=torch.float32)`	Criar a partir de listas ou arrays (cópia).
`t.shape  t.dtype  t.device`	Forma, tipo e onde está (CPU/GPU).
`t.sum(dim=0)  t @ u  t.T`	`dim` em vez de `axis`.
`torch.from_numpy(a)  t.numpy()`	Converter sem copiar: partilham a memória.
`t.to("cuda")  modelo.to("cuda")`	GPU, se `torch.cuda.is_available()`.

08 AUTOGRAD

N3_05 · §2

`torch.tensor(0.5, requires_grad=True)`	Registar as operações (grafo de cálculo).
`L.backward()  w.grad`	Calcula todos os gradientes; ficam em .grad.
`w.grad.zero_()`	Os gradientes somam-se a cada backward(): pôr a zero.
`with torch.no_grad():  t.detach()`	Sem registo: avaliar, prever, mexer em pesos.

09 DEFINIR A REDE

N3_05 · §3

`class Rede(nn.Module):`
`  def __init__(self):`
`  super().__init__()`
`  self.c1 = nn.Linear(2, 16)`
`  self.c2 = nn.Linear(16, 1)`
`  def forward(self, x):`
`  x = torch.relu(self.c1(x))`
`  return self.c2(x)`
	Camadas no `__init__`, cálculo no `forward`. O `nn.Linear` guarda os pesos transpostos: (saídas, entradas).

`nn.Sequential(nn.Linear(2, 16),` `  nn.ReLU(), nn.Linear(16, 1))`	A forma curta, para camadas em sequência.

10 PERDAS E OTIMIZADORES

N3_05 · §3

Tarefa	Saída	Perda
binária	1 *logit*	`nn.BCEWithLogitsLoss()`
várias classes	1 *logit* por classe	`nn.CrossEntropyLoss()`
regressão	1 valor	`nn.MSELoss()`

`torch.optim.SGD(m.parameters(),` `  lr=0.1, momentum=0.9)`	Descida do gradiente (com momentum).
`torch.optim.Adam(m.parameters(),` `  lr=1e-3, weight_decay=1e-4)`	Adam: bom ponto de partida. weight_decay = L2.

11 DADOS E CICLO DE TREINO

N3_05 · §4–5

`DataLoader(TensorDataset(X_t, y_t),` `  batch_size=32, shuffle=True)`	Mini-lotes. Ou um Dataset próprio (`__len__`, `__getitem__`).

`modelo.train()`
`for X, y in carregador:`
`  perda = perda_fn(modelo(X), y)`
`  otimizador.zero_grad()`
`  perda.backward()`
`  otimizador.step()`
	As cinco linhas de cada passo de treino.

`modelo.eval()`
`with torch.no_grad():`
`  p = torch.sigmoid(modelo(X))`
	Avaliar: `eval()` desliga o dropout; `no_grad` poupa memória.

`torch.save(m.state_dict(), "m.pt")`
`m.load_state_dict(torch.load(`
`  "m.pt", weights_only=True))`
	Guardar e carregar os pesos.

! ARMADILHAS

float64 vs float32	Arrays do NumPy chegam em float64: converter com `.float()`.
Esquecer zero_grad()	Os gradientes acumulam-se entre passos.
Dados tabulares	Comparar sempre com regressão logística e árvores: a rede raramente ganha.